Phase 1H — Hardening & Production Readiness

Phase 1H makes BaseCenter production-ready by adding transactional email, audit logging, rate limiting, and security hardening. Every new capability **degrades gracefully when its dependency is not configured** — mirroring the Stripe "graceful when unconfigured" pattern established in earlier phases. In particular, when no SMTP server is configured the platform keeps working: verification, reset, and invitation links are written to the server log instead of being emailed, so no user flow ever breaks.

> Scope note: this phase is **code hardening only** — it does not change hosting or infrastructure. It runs on the same native PostgreSQL + FastAPI + Next.js stack as prior phases.

Part A — Transactional email

1. Email service abstraction

A new `email_service` (`backend/app/services/email_service.py`) centralizes all outbound mail:
- `is_configured()` — true only when host, port, and from-address are set.
- `send_email(...)` — sends via `smtplib` (STARTTLS/SSL per settings) when configured; otherwise **logs the message and any action link** via the `basecenter.email` logger and returns without error.
- Purpose-specific helpers: `send_verification_email` , `send_password_reset_email` , `send_invite_email` , `send_test_email` .
- **Every send always logs the action link** so links are recoverable from the server log even when SMTP is configured.

SMTP credentials are stored through the existing encrypted platform-settings service (`EMAIL_SMTP_PASSWORD` is marked `is_secret=True` and encrypted at rest; it is never returned in plaintext — responses mask it with `••••••••`).

2. Super Admin — Email Settings page

A new **Email Settings** page (`/admin/settings/email`) modeled on the Stripe settings page:
- Fields: from name, from address, SMTP host, port, username, password (write-only; blank leaves the saved value unchanged), and TLS toggle.
- A **delivery status** card shows "SMTP configured" vs. "Not configured — links are logged to the server".
- **Send a test email** to any recipient. When SMTP is unconfigured the test succeeds as `{sent:false, logged:true}` and the content is logged.

3. Wired into real flows with signed tokens

Signed, expiring tokens are minted with `itsdangerous` (`backend/app/core/tokens.py`) for three purposes:

Purpose	Default expiry
Email verification	48 hours
Password reset	2 hours
Team invitation	168 hours (7 days)

- **Signup** now sends a verification email and marks the account `email_verified=false` until the link is used.
- **User invitation** (team page): a tenant admin can add a member **without** setting a password — an invitation email is sent with a link to set their own password. When SMTP is unconfigured the API returns the invite link directly so the admin can share it manually.
- **Password reset**: request → email with reset link → confirm sets the new password. The confirm endpoint accepts **both** reset and invite tokens, so invited users use the same "set password" flow.

New tenant-portal pages: **/app/verify-email**, **/app/forgot-password**, and **/app/reset-password** (the token-reading pages wrap `useSearchParams` in `<Suspense>` for the Next.js build). A **"Forgot password?"** link was added to the tenant login page.

Part B — Audit logging

A new `audit_logs` table + `audit_service` records security-relevant actions. `audit_service.record(...)` is **fail-safe**: any error is swallowed and rolled back so auditing never breaks the primary request.

Instrumented actions include:
- Super Admin login
- Tenant create / suspend / reactivate
- Seat (quantity) changes
- Module enable / disable
- Stripe, sidebar-label, and email settings changes
- Subscription cancel / reactivate

Each entry captures actor (user id, email, role), action, target, free-form metadata, IP address, and timestamp.

Super Admin — Audit Log page

A new **Audit Log** page (`/admin/audit`) provides a paginated, filterable table (filter by actor email, action, and date range). The endpoint (`GET /api/v1/admin/audit`) is protected by `get_current_superuser`.

Part C — Rate limiting

`slowapi` limits abuse-prone endpoints (keyed by client IP, returning **HTTP 429** with a JSON body). Limits are configurable via environment variables:

Endpoint	Default limit	Env var
SA + tenant login	10 / minute	`RATE_LIMIT_LOGIN`
Signup	5 / minute	`RATE_LIMIT_SIGNUP`
Password reset (request/confirm)	5 / minute	`RATE_LIMIT_PASSWORD_RESET`
Send test email	5 / minute	`RATE_LIMIT_TEST_EMAIL`

Rate limiting can be globally toggled with `RATE_LIMIT_ENABLED`.

Part D — Security hardening

- **Security headers** on every response via middleware: `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `X-XSS-Protection`, `Permissions-Policy`, and a `Content-Security-Policy` with `frame-ancestors 'self' https://*.abacus.ai`. **No restrictive `X-Frame-Options` is set**, so the app remains embeddable in the Abacus preview iframe.
- **Explicit CORS** configuration.
- **Password strength** enforced on signup and password reset/confirm (minimum 8 characters, at least one letter and one digit; HTTP 400 on failure) via `backend/app/core/passwords.py`.
- **No secrets logged** — SMTP passwords and Stripe keys are never written to logs; secret settings are encrypted at rest and masked in API responses.

Files

Backend — new

- `app/models/audit_log.py` — `AuditLog` model.
- `alembic/versions/f4d5e6a7b8c9_phase1h_hardening.py` — migration (`users.email_verified` + `audit_logs`).
- `app/core/tokens.py` — signed token mint/verify.
- `app/core/passwords.py` — password strength validator.
- `app/core/rate_limit.py` — slowapi limiter + 429 handler.
- `app/services/email_service.py` — email abstraction (send or log).
- `app/services/audit_service.py` — fail-safe audit recorder + query.

- `app/schemas/email_settings.py`, `app/schemas/audit.py`, `app/schemas/auth_extra.py` — request/response schemas.

Backend — modified

- `app/core/config.py` — token expiry + rate-limit settings.
- `app/models/user.py` — `email_verified` column.
- `app/services/settings_service.py` — email config get/update (encrypted, masked).
- `app/main.py` — logger setup, rate-limit middleware/handler, security headers.
- `app/api/v1/auth.py` — SA login rate limit + audit.
- `app/api/v1/admin.py` — email endpoints, audit endpoint, audit instrumentation.
- `app/api/v1/tenant.py` — signup/login/reset/verify/invite flows + audit.
- `app/schemas/tenant_portal.py` — optional invite password; invite response.
- `requirements.txt` — `slowapi`, `itsdangerous`.

Frontend — new

- `app/admin/settings/email/page.tsx` — Email Settings page.
- `app/admin/audit/page.tsx` — Audit Log page.
- `app/app/forgot-password/page.tsx`, `app/app/reset-password/page.tsx`, `app/app/verify-email/page.tsx` — tenant portal flows.

Frontend — modified

- `lib/api.ts` — email/audit/password/verify API functions and types.
- `components/admin/AdminShell.tsx` — Email Settings + Audit Log nav items.
- `app/app/login/page.tsx` — “Forgot password?” link.
- `app/app/team/page.tsx` — optional password / email-invitation flow.

Verification

- `alembic upgrade head` applies `f4d5e6a7b8c9`; `users.email_verified` and `audit_logs` verified in PostgreSQL.
- Backend imports and registers all routes; `npx tsc --noEmit` and `npm run build` both pass, building the new `/admin/audit`, `/admin/settings/email`, `/app/forgot-password`, `/app/reset-password`, and `/app/verify-email` routes.
- End-to-end smoke test (API, SMTP unconfigured):
- Security headers present; **no** `X-Frame-Options: DENY`; CSP `frame-ancestors` correct (iframe embedding preserved).
- Email settings get/update; test email returns `{sent:false, logged:true}`.
- Signup logs a verification link; **weak password rejected** (400).
- Password reset: request → link logged → confirm → old password fails, new password works.
- Verify-email marks the account verified; a bad token is rejected.
- Invitation with no password → `invited=true` with an invite link; link logged.
- Audit list works, including filtering by action (counts correct).
- **Rate limiting**: 14 rapid login attempts → 401 × 9 then **429 × 5**.

Summary

Capability	Mechanism
Transactional email	`email_service`: SMTP when configured, else logs the link
Email settings	SA page + encrypted, masked SMTP credentials + test send
Email verification	Signed 48h token; `/app/verify-email`
Password reset	Signed 2h token; forgot/reset pages; confirm accepts invites
User invitation	Optional password → 7-day invite token + email (or shown link)
Audit logging	`audit_logs` + fail-safe recorder; SA filterable page
Rate limiting	slowapi on login/signup/reset/test-email; 429; env-configurable
Security headers	nosniff, Referrer/Permissions-Policy, CSP frame-ancestors
iframe embedding	Preserved — CSP `frame-ancestors`, no `X-Frame-Options`
Password strength	Min 8 chars incl. letter + digit on signup/reset
Graceful degradation	No SMTP → links logged / returned; nothing breaks